

Geneva Chunker-MV Refresh: Memory Fix Specification

cantina-tools | geneva==0.13.0b9 | file: geneva/runners/ray/pipeline.py (unless noted) | 2026-06-09

The defect

A chunker materialized-view refresh expands each work item **entirely in actor memory before any output is emitted**. The per-actor peak is the full expansion of up to 1024 source rows, held simultaneously in three representations:

- **Python lists for the whole batch** — `Chunker._execute_scalar` accumulates every output row into `output_cols` across all source rows, then builds one `RecordBatch` (transformer.py:2374; loop :2409; batch :2440).
- **One returned Arrow batch** — `execute_on_record_batch` returns that single batch with all rows (transformer.py:2351).
- **One serialized IPC blob** — `ChunkerExpandActor.expand_batch` rebuilds it as `result_batch` and serializes the whole thing to a byte buffer (pipeline.py:3791; build :3849; serialize :3855–3862).

The work-item size is the parallelism unit *and* the expansion unit, fixed at `row_id_batch_size = 1024` (:3959; batched :3988). Peak actor RAM = $1024 \times \text{fanout} \times \text{bytes-per-output-row}$, independent of every currently exposed tunable.

Why max_rows_per_fragment does not bound it

`max_rows_per_fragment` (exposed to us as `checkpoint_size`) is applied **only on the driver, after the actor returns the whole blob**: the driver reads the full result back (`reader.read_all()`, :4099) and then slices it into fragments (:4110–4118). It governs output-fragment granularity, not actor memory; nothing on the actor path consults it.

The fix

Bound the actor's expansion unit by output-row count and flush incrementally, instead of expanding the whole work item and returning one blob. Two coordinated changes:

1. **Add a streaming execution path to Chunker** (transformer.py). Introduce `execute_on_record_batch_iter(record_batch, max_rows) → Iterator[RecordBatch]`: the existing per-source-row loop (:2409), modified to **yield the accumulated sub-batch whenever the output-row count reaches max_rows**, then reset and continue. The current `execute_on_record_batch` becomes a thin wrapper that concatenates the iterator, preserving the existing API.
2. **Make expand_batch write fragments incrementally and return metadata** (pipeline.py:3791). Replace the single-shot expand/serialize with a loop over the iterator from (1): for each bounded sub-batch, do the inherited-column join (logic now at :3826–3851) and **write a destination fragment directly** via `LanceFragment.create()`, accumulating only lightweight `FragmentMetadata`. Return the metadata list instead of `result_ipc`; the driver collect loop (:4084–4118) gathers metadata and commits rather than reading blobs back and slicing.

Precedent in-tree: the non-chunker UDTF path (`_refresh_udtf_matview/_run_udtf_refresh`) already has actors write data files via `LanceFragment.create()` and return `FragmentMetadata` JSON, with the driver doing a single `LanceOperation.Overwrite + commit`. The chunker path should adopt the same shape.

Net effect: peak actor RAM drops from $1024 \times \text{fanout} \times S$ to $\text{max_rows} \times S$ plus one source row's transient decode working set — bounded by construction, for any payload size, using the knob that already exists.

Smaller-diff alternative & an orthogonal knob

- **Streaming-generator variant**. Keep the driver as writer but convert `expand_batch` to a Ray streaming generator (`num_returns="streaming"`), yielding one $\leq \text{max_rows}$ IPC sub-batch at a time; the driver `add()`s each as it arrives. Bounds the actor buffer to one sub-batch, but bytes still transit the driver serially — strictly weaker than writing fragments from the actor, smaller change.
- **Expose row_id_batch_size**. Surface the hard-coded 1024 (:3959) as a refresh parameter. Not a fix on its own, but lets operators shrink the work item B to gain parallelism and lower memory as a stopgap — a one-line plumbing change.

Correctness considerations

Concern	Handling
---------	----------

Ordering / IDs	<code>__source_row_id/ __child_index</code> are assigned per source row (transformer.py:2411). Flushing on output count may split one source row's children across fragments — safe; rows are independent and the only guarantee is deterministic order within a partition, preserved by processing <code>row_ids</code> in order.
<code>output_limit</code>	Currently enforced during the driver slice (:4103–4108); must move into the streaming consumer so it can stop mid-iteration.
Checkpoint / resume	Today a work item checkpoints as a unit. N fragments per work item changes the resume granularity; <code>format_udtf_fragment_key</code> and the driver's metadata collection must account for multiple fragments per item.
Error handling	The <code>skip_on_error</code> path (:2426) stays per source row — unaffected.
Empty results	Preserve the <code>out_rows == 0 → (None, ...)</code> short-circuit (:3824) so all-empty work items commit nothing.